

Phan 8.6 - Generic Method trong C#

Giai thích toan tap: tu van de thuc te den ung dung trong .NET

Mục tiêu: Hieu Generic không chỉ ở cú pháp <T>, mà phải thay rõ Generic giải quyết vấn đề gì, khi nào nên dùng và T hoạt động ra sao.

1. Van de Generic giải quyết

Gia su ban can mot method lay phan tu dau tien trong danh sach.

```
public static int GetFirstNumber(List<int> numbers)
{
    return numbers[0];
}
```

Voi string:

```
public static string GetFirstName(List<string> names)
{
    return names[0];
}
```

Voi Student:

```
public static Student GetFirstStudent(List<Student> students)
{
    return students[0];
}
```

Ba method tren co cung mot logic: nhan danh sach -> lay items[0] -> tra ve phan tu. Chi co kieu du lieu thay doi: int, string, Student.

Day la luc Generic co ich. Ta viet mot lan:

```
public static T GetFirst<T>(List<T> items)
{
    return items[0];
}
```

Sau do co the dung cho List<int>, List<string>, List<Student>, List<Product>, List<Order>...

Y quan trong nhất: Generic cho phép viết một logic một lần và sử dụng lại với nhiều kiểu dữ liệu, nhưng vẫn giữ an toàn kiểu (type safety).

2. T thực sự là gì?

Trong public static T GetFirst<T>(List<T> items), T là type parameter - tham số đại diện cho kiểu dữ liệu.

Nó giống parameter bình thường, nhưng thay vì nhận một giá trị, nó đại diện cho một kiểu.

```
public static void Print(string name)
```

Parameter name có thể nhận các giá trị như "Hung", "An", "Binh".

```
public static void Print<T>(T value)
```

Con T có thể được thay bằng int, string, Student, Product... khi method được gọi.

3. Cách hình dung việc thay thế T

```
public static T GetFirst<T>(List<T> items)
{
    return items[0];
}

List<int> numbers = new List<int> { 10, 20, 30 };
int number = GetFirst(numbers);
```

C# nhìn thấy List<int> nên suy luận T = int. Về mặt tu duy, bạn có thể hình dung method lúc này giống như:

```
public static int GetFirst(List<int> items)
{
    return items[0];
}
```

Với List<string>, T = string. Với List<Student>, T = Student. Đây chỉ là cách hình dung để học; compiler không cần phải viết lại source code của bạn như vậy.

4. Cấu pháp Generic Method

```
accessModifier returnType MethodName<T>(parameters)
{
}
```

Ví dụ:

```
public static void Display<T>(T value)
{
    Console.WriteLine(value);
}
```

Thành phần	Ý nghĩa
public	Phạm vi truy cập
static	Có thể gọi bằng tên class
void	Không trả về giá trị
Display	Tên method
<T>	Khai báo type parameter T
T value	Parameter value có kiểu T

Đề note: Dấu <T> sau tên method là nơi khai báo type parameter. Nếu bỏ <T>, compiler không biết T là gì.

5. T không bắt buộc phải tên là T

```
public static void Display<TValue>(TValue value)
{
    Console.WriteLine(value);
}
```

Quy ước đặt tên thường gặp:

Tên	Ý nghĩa thường dùng
T	Kiểu chung
TItem	Kiểu phần tử
TKey	Kiểu khóa
TValue	Kiểu giá trị
TResult	Kiểu kết quả
TEntity	Entity

Bạn đã gặp Dictionary<TKey, TValue>. Đây cũng là một ví dụ về nhiều type parameter.

6. Type inference - C# tự suy luận T

```
public static void Display<T>(T value)
{
    Console.WriteLine(value);
}

Display<int>(10);
Display<string>("Hello");
```

Thông thường có thể viết gọn:

```
Display(10);    // T = int
Display("Hello"); // T = string
Display(5.5m); // T = decimal
```

C# nhìn vào argument để suy ra T. Đó là type inference.

7. Generic method nhận một parameter kiểu T

```
public static void DisplayValue<T>(T value)
{
    Console.WriteLine(value);
}

DisplayValue(10);
DisplayValue("C#");
DisplayValue(9.5);
```

Một method có thể được gọi với nhiều kiểu khác nhau mà không cần viết nhiều overload trung logic.

8. Hien thi kieu that cua T bang typeof(T)

```
public static void DisplayValue<T>(T value)
{
    Console.WriteLine(
        $"Type: {typeof(T).Name}, Value: {value}");
}

DisplayValue(10);
DisplayValue("Hello");
DisplayValue(1_000_000m);
```

Ket qua co the la:

```
Type: Int32, Value: 10
Type: String, Value: Hello
Type: Decimal, Value: 1000000
```

9. Generic method nhan nhieu parameter cung kieu

```
public static void DisplayPair<T>(T first, T second)
{
    Console.WriteLine(first);
    Console.WriteLine(second);
}

DisplayPair(10, 20);    // T = int
DisplayPair("C#", ".NET"); // T = string
```

Neu goi DisplayPair(10, "Hello"), C# co the khong suy luan duoc mot T duy nhat vi hai argument co kieu khac nhau.

10. Generic method tra ve T

```
public static T GetFirst<T>(List<T> items)
{
    return items[0];
}
```

Neu dau vao la List<int> thi ket qua la int. Neu la List<string> thi ket qua la string. Neu la List<Student> thi ket qua la Student.

Y nghĩa: Dua vao danh sach kieu nao thi method tra ra mot phan tu dung kieu do. Compiler giu duoc type safety tu dau vao den dau ra.

11. GetFirst<T> co validation

```
public static T GetFirst<T>(List<T> items)
{
    if (items == null)
    {
        throw new ArgumentNullException(
            nameof(items),
            "The list cannot be null.");
    }

    if (items.Count == 0)
    {
        throw new InvalidOperationException(
            "The list is empty.");
    }
}
```

```

    }

    return items[0];
}

```

Can kiem tra ca `items == null` va `items.Count == 0` truoc khi truy cap `items[0]`.

12. GetLast<T>

```

public static T GetLast<T>(List<T> items)
{
    if (items == null)
    {
        throw new ArgumentNullException(nameof(items));
    }

    if (items.Count == 0)
    {
        throw new InvalidOperationException(
            "The list is empty.");
    }

    return items[items.Count - 1];
}

List<int> numbers = new() { 10, 20, 30 };
int last = GetLast(numbers); // 30

```

13. Generic method voi array

Generic khong chi dung voi `List<T>`.

```

public static T GetLast<T>(T[] items)
{
    if (items == null)
    {
        throw new ArgumentNullException(nameof(items));
    }

    if (items.Length == 0)
    {
        throw new InvalidOperationException(
            "Array is empty.");
    }

    return items[items.Length - 1];
}

int[] numbers = { 1, 2, 3 };
int result = GetLast(numbers);

string[] names = { "Hung", "An", "Binh" };
string lastName = GetLast(names);

```

14. Swap<T> - vi du thay ro cong nang Generic

Neu khong co Generic, ban co the phai viet `SwapInt()`, `SwapString()`, `SwapStudent()` du logic doi cho hoan toan giong nhau.

```
public static void Swap<T>(ref T first, ref T second)
{
    T temp = first;
    first = second;
    second = temp;
}

int a = 10;
int b = 20;
Swap(ref a, ref b);
// a = 20, b = 10

string first = "Hung";
string second = "An";
Swap(ref first, ref second);
// first = "An", second = "Hung"
```

15. Vì sao Swap cần ref?

Nếu method nhận T first, T second theo cách bình thường, việc gán lại hai biến chỉ thay đổi bản sao local bên trong method. ref cho phép method tác động trực tiếp lên biến được truyền vào.

```
Swap(ref a, ref b);
```

Phân biệt: ref là cơ chế cho phép thay đổi biến bên ngoài. Generic chỉ giúp Swap dùng được với nhiều kiểu T. Hai khái niệm này có vai trò khác nhau.

16. Một method có thể có nhiều type parameter

```
public static void DisplayKeyValue<TKey, TValue>(
    TKey key,
    TValue value)
{
    Console.WriteLine(
        $"Key: {key}, Value: {value}");
}

DisplayKeyValue("SV01", "Hung");
// TKey = string, TValue = string

DisplayKeyValue("SP01", 10_000_000m);
// TKey = string, TValue = decimal

DisplayKeyValue(1, student);
// TKey = int, TValue = Student
```

Đây là cùng tu duy mà Dictionary<TKey, TValue> sử dụng.

17. Generic method không bắt buộc trả về T

```
public static bool AreEqual<T>(T first, T second)
{
    return EqualityComparer<T>
        .Default
        .Equals(first, second);
}

bool result1 = AreEqual(10, 10);
bool result2 = AreEqual("C#", ".NET");
```

Parameter có kiểu T, nhưng return type có thể là bool, int, string hoặc bất kỳ kiểu phù hợp với logic.

18. Vì sao không viết `first == second`?

Với một `T` hoàn toàn tổng quát, compiler không thể mặc định mọi kiểu `T` đều hỗ trợ toán tử `==` theo cách bạn muốn sử dụng. Một cách tổng quát hơn là `EqualityComparer<T>.Default.Equals(first, second)`.

```
return EqualityComparer<T>.Default.Equals(
    first,
    second);
```

19. Generic Method và Method Overload

Overload viết nhiều method cùng tên nhưng parameter khác kiểu:

```
public static void Display(int value) { }
public static void Display(string value) { }
public static void Display(double value) { }
```

Generic có thể gộp lại khi logic giống nhau:

```
public static void Display<T>(T value)
{
}
```

Tình huống	Nội dung
Logic giống nhau, chỉ khác kiểu dữ liệu	Generic
Mọi kiểu cần cách xử lý khác nhau	Overload

Ví dụ logic khác nhau: định dạng `int` theo một cách, định dạng `DateTime` theo `dd/MM/yyyy`. Khi đó overload có thể phù hợp hơn.

20. Generic khác object như thế nào?

object có thể nhận nhiều kiểu, nhưng thường làm mất thông tin kiểu cụ thể và có thể cần cast lại.

```
object result = GetValue();
Student student = (Student)result;
```

Generic giữ liên kết kiểu:

```
Student student = GetFirst(students);
```

Với `List<Student>`, compiler suy luận `T = Student` và biết kết quả là `Student`.

21. Ba lợi ích lớn của Generic

1. Tài sử dụng code: một `GetFirst<T>()` thay cho `GetFirstInt()`, `GetFirstString()`, `GetFirstStudent()`...
2. Type safety: compiler giữ đúng kiểu từ đầu vào đến đầu ra.
3. Giám ép kiểu thu công: không cần cast một object về kiểu mong muốn trong nhiều tình huống.

22. Generic ban da su dung tu truoc

```
List<int>
List<Student>
Queue<Order>
Stack<string>
HashSet<int>
Dictionary<string, Product>
```

Tat ca deu la Generic. Khac biet o day: List<T>, Queue<T>, Stack<T> la generic class; GetFirst<T>() la generic method.

23. Han che quan trong cua T

Vi compiler chua biet T cu the la gi, ban khong the tuy y su dung moi phep toan.

```
public static T Add<T>(T first, T second)
{
    return first + second; // khong dam bao hop le voi moi T
}
```

T co the la int, decimal, string, Student, Order... Khong phai tat ca cac kieu deu co cung cac toan tu +, -, *, /, >, <.

24. Khong the tuy y goi method riêng tren T

```
public static void Print<T>(T value)
{
    value.DisplayInfo(); // compiler khong biet T co method nay
}
```

Neu T = int, int khong co DisplayInfo(). De yeu cau T phai co mot dac diem nao do, ta can Generic Constraint.

25. Khong the tuy y tao new T()

```
public static T Create<T>()
{
    return new T();
}
```

Compiler khong chac T co constructor khong tham so. Neu can, ta co the dung constraint:

```
public static T Create<T>()
    where T : new()
{
    return new T();
}
```

where T : new() se duoc hoc ky o Phan 8.8.

26. Generic Constraint dung de lam gi?

Constraint nghĩa là: tôi chấp nhận Generic, nhưng T phải đáp ứng một điều kiện.

```
where T : class
where T : new()
where T : Employee
where T : IPrintable
```


Constraint	Y nghĩa
where T : class	T phải là reference type
where T : new()	T phải có constructor không tham số
where T : Employee	T phải là Employee hoặc lớp con
where T : IPrintable	T phải implement IPrintable

O 8.6, chỉ cần hiểu mục đích. Phan 8.8 sẽ học sau.

27. Generic Method có thể nằm trong class thường

```
public static class GenericHelper
{
    public static void Display<T>(T value)
    {
        Console.WriteLine(value);
    }
}

GenericHelper.Display(10);
```

28. Generic Method cũng có thể là instance method

```
public class Printer
{
    public void Display<T>(T value)
    {
        Console.WriteLine(value);
    }
}

Printer printer = new Printer();
printer.Display(10);
printer.Display("Hello");
```

Nếu method không cần trạng thái của object, static thường gọn và phù hợp hơn.

29. Generic Class và Generic Method khác nhau

```
// Generic Method
public static T GetFirst<T>(List<T> items)
{
    return items[0];
}

// Generic Class
public class Storage<T>
{
    private T _value;
}
```

Dạng	Phạm vi của T
Generic Method	T thuộc về method đó

Dang	Pham vi cua T
Generic Class	T duoc dung o cap class va cac member cua class

Generic Class se la Phan 8.7.

30. Generic Method trong .NET thuc te

JSON

```
Student? student =
    JsonSerializer.Deserialize<Student>(json);

Product? product =
    JsonSerializer.Deserialize<Product>(json);
```

Cung mot method `Deserialize<T>`, nhưng T co the la Student, Product, Order...

Entity Framework Core

```
dbContext.Set<Student>();
dbContext.Set<Product>();
```

Dependency Injection

```
services.AddScoped<
    IProductService,
    ProductService>();
```

LINQ

```
Where<T>()
First<T>()
ToList<T>()
```

Async

```
Task<Student>
Task<Product>
Task<List<Student>>
```

Nhưng vì dụ này cho thấy Generic là một phần rất phổ biến của .NET, không chỉ là kiến thức lý thuyết.

31. Cách nhận biết khi nào nên dùng Generic

Câu hỏi nên tu dat: Mình có đang viết lại cùng một logic chỉ vì kiểu dữ liệu khác nhau không?

Nếu có, Generic có khả năng phù hợp.

```
GetFirstInt
GetFirstString
GetFirstStudent

// -> logic giống nhau, nên nghĩ đến Generic

SwapInt
SwapString
SwapStudent

// -> logic giống nhau, nên nghĩ đến Generic
```

Nhưng `CalculateStudentGpa()` và `CalculateEmployeeSalary()` có logic khác nhau, nên không phải ví dụ Generic đơn giản chỉ vì chúng cùng trả về một số.

32. Ví dụ hoàn chỉnh - `GenericHelper`

```
public static class GenericHelper
{
    public static void DisplayValue<T>(T value)
    {
        Console.WriteLine(
            $"Type: {typeof(T).Name}, Value: {value}");
    }

    public static T GetFirstItem<T>(List<T> items)
    {
        if (items == null)
        {
            throw new ArgumentNullException(
                nameof(items),
                "The list cannot be null.");
        }

        if (items.Count == 0)
        {
            throw new InvalidOperationException(
                "The list cannot be empty.");
        }

        return items[0];
    }

    public static T GetLastItem<T>(List<T> items)
    {
        if (items == null)
        {
            throw new ArgumentNullException(
                nameof(items),
                "The list cannot be null.");
        }

        if (items.Count == 0)
        {
            throw new InvalidOperationException(
                "The list cannot be empty.");
        }

        return items[items.Count - 1];
    }

    public static void SwapValues<T>(ref T first, ref T second)
    {
        T temp = first;
        first = second;
        second = temp;
    }

    public static bool AreValuesEqual<T>(T first, T second)
    {
        return EqualityComparer<T>
            .Default
            .Equals(first, second);
    }
}
```

```
public static void DisplayKeyValue<TKey, TValue>(
    TKey key,
    TValue value)
{
    Console.WriteLine(
        $"Key: {key}, Value: {value}");
}
}
```

33. Phan tich GetFirstItem<T>

```
public static T GetFirstItem<T>(List<T> items)
```

Co ba vi tri can doc duoc:

- <T>: khai bao method co type parameter T.
- List<T>: danh sach chua phan tu kieu T.
- T truoc GetFirstItem: return type cung la T.

Dich sang tieng Viet: Nhan mot danh sach chua kieu gi thi tra ve mot phan tu dung kieu do.

34. Phan tich SwapValues<T>

```
public static void SwapValues<T>(
    ref T first,
    ref T second)
```

Co the doc: nhan hai bien cung mot kieu bat ky va doi gia tri cua chung. T co the la int, string, Student...

35. Phan tich <TKey, TValue>

```
public static void DisplayKeyValue<TKey, TValue>(
    TKey key,
    TValue value)
```

Method can hai kieu doc lap. Kieu key khong bat buoc giong kieu value.

```
DisplayKeyValue("SP01", 500000m);
// TKey = string
// TValue = decimal
```

36. Loi thuong gap 1 - Quen khai bao <T>

Sai:

```
public static T GetValue(T value)
{
    return value;
}
```

Dung:

```
public static T GetValue<T>(T value)
{
    return value;
}
```

37. Loi thuong gap 2 - Khai bao <T> nhưng không dùng

```
public static void Display<T>(string value)
{
    Console.WriteLine(value);
}
```

T không tham gia vào method nên Generic không có ý nghĩa. Hoặc bỏ <T>, hoặc để parameter có kiểu T nếu logic thực sự cần Generic.

38. Loi thuong gap 3 - Gia dinh T có chức năng nào đó

```
public static void Print<T>(T item)
{
    item.DisplayInfo();
}
```

Compiler không biết T có DisplayInfo(). Muốn đảm bảo, cần constraint với class cha hoặc interface phù hợp.

39. Loi thuong gap 4 - Không validation collection

```
return items[0];
```

Nếu items null hoặc rỗng, method sẽ lỗi. Hãy kiểm tra null và Count trước khi truy cập phần tử.

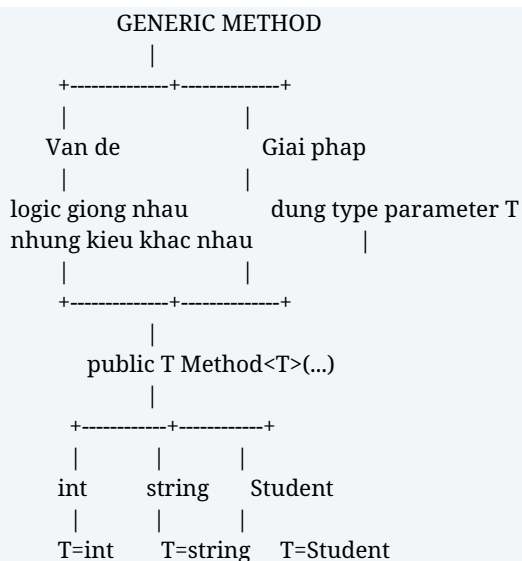
40. Loi thuong gap 5 - Dùng Generic khi không cần

```
public static void SayHello<T>()
{
    Console.WriteLine("Hello");
}
```

T không được dùng. Method chỉ cần:

```
public static void SayHello()
{
    Console.WriteLine("Hello");
}
```

41. So do tu duy toan phan



42. Mối quan hệ với các collection đã học

Bạn đã học `List<T>`, `Dictionary<TKey,TValue>`, `HashSet<T>`, `Queue<T>`, `Stack<T>`. Tất cả đều dựa trên Generic.

Bước chuyển hiện tại: Trước đây bạn sử dụng Generic do .NET viết sẵn. Ở Phan 8.6, bạn bắt đầu tự viết Generic Method của mình.

43. Những gì cần thực sự thuộc

- Generic Method giải quyết trường hợp: logic giống nhau + kiểu dữ liệu khác nhau.
- Khai báo có dạng `public static void Display<T>(T value)`.
- Method có thể trả về T, ví dụ `public static T GetFirst<T>(List<T> items)`.
- C# có thể tự suy luận T từ argument, ví dụ `Display(10) -> T = int`.
- Có thể có nhiều type parameter, ví dụ `<TKey, TValue>`.
- Không được mặc định T có toán tử, method riêng hay constructor. Muốn yêu cầu phải dùng constraint.

44. Câu kiểm tra nhanh trước khi làm bài

1. Vì sao cần Generic?

Tra lời: Để tái sử dụng cùng một logic cho nhiều kiểu dữ liệu nhưng vẫn đảm bảo type safety.

2. Trong `GetFirst<T>()`, T là gì?

Tra lời: Là type parameter đại diện cho kiểu mà khi method được gọi.

3. Nếu truyền `List<Student>` vào `GetFirst<T>()` thì T là gì?

Tra lời: `T = Student`.

4. Generic Method khác Generic Class thế nào?

Tra lời: Generic Method chỉ khai báo type parameter cho method; Generic Class khai báo type parameter ở cấp class.

5. Vì sao không dùng object hết?

Tra lời: Generic giữ được kiểu cụ thể và thường không cần cast, trong khi object có thể làm mất thông tin kiểu cụ thể.

6. Vì sao không thể tùy ý viết `first + second`?

Tra lời: Vì compiler không biết mọi T đều hỗ trợ toán tử `+`.

Ket luan

Cau can nho: Generic Method la method co type parameter nhu T, cho phep cung mot logic hoat dong voi nhieu kieu du lieu ma van giu an toan kieu.

Cach nhan biet nhanh: Cung logic + chi khac kieu du lieu -> hay nghi den Generic.

Sau khi nam chac phan nay, bai tap GenericHelper se giup bien cac khai niem thanh phan xa code.